

SmartParking

Documentazione del progetto – Testing e Verifica del Software

Francesco Tomasoni

A.A. 2025/2026

Contents

1	Introduzione e requisiti	3
1.1	Descrizione del sistema	3
1.2	Requisiti funzionali	3
1.3	Requisiti non funzionali	4
1.4	Macchina a stati	4
1.5	Glossario	4
2	Modello Asmeta	5
2.1	Il modello (<code>asmeta/src/SmartParking.asm</code>)	5
2.1.1	Simulazione manuale	6
2.2	Scenari Avalla	6
2.3	AsmetaV – validazione con copertura (Vc)	7
2.4	Model checking (<code>asmeta/src/SmartParking_MC.asm</code>)	8
2.4.1	Model Advisor (opzionale)	9
2.4.2	ATGT – generazione automatica di scenari (opzionale)	10
3	Implementazione Java	10
3.1	Struttura del progetto Maven	10
3.2	La macchina a stati	13
3.3	Interfaccia utente (Vaadin)	13
3.4	Test end-to-end con Selenium	13
4	Testing Java	13
4.1	Struttura dei test	13
4.2	MC/DC	15
4.3	Combinatorial testing (pairwise)	15
4.4	Mockito	16
4.5	Model Based Testing	16
4.6	Esecuzione e copertura	16
5	JML e verifica statica (ESC)	17
5.1	Invarianti	19
5.2	Costruttori	19
5.3	Metodo <code>assegnaPosto</code>	20
5.4	Metodo <code>liberaPosto</code>	21
5.5	Metodi getter	21
5.6	Verifica dei contratti con OpenJML (ESC)	21

6	Analisi statica e ispezione del codice con LLM	22
6.1	Checklist preliminare	23
6.2	Osservazioni emerse e come sono state gestite	23
6.3	Bilancio dell'ispezione	24
7	Continuous Integration	24
8	Conclusioni	25
8.1	Riepilogo delle evidenze	25

1 Introduzione e requisiti

1.1 Descrizione del sistema

SmartParking è un sistema che gestisce in automatico un parcheggio con due sbarre, una in ingresso e una in uscita, dei sensori che rilevano le auto e un totem che riconosce il tipo di utente. I posti sono di due categorie, standard e riservati ai disabili; nel modello, per semplicità, ne ho tenuto uno solo per categoria.

Gli utenti che il sistema riconosce sono tre. L'utente STD è quello occasionale, che paga la sosta. Il DISABILE ha il permesso, non paga ed ha la priorità sui posti riservati. L'ABBONATO ha l'abbonamento mensile e, come il disabile, non paga. Quando arriva un'auto il sistema rileva la richiesta, capisce che tipo di utente è e controlla se c'è un posto adatto prima di alzare la sbarra. Se il posto non c'è, l'accesso viene negato.

Il disabile, come detto, ha la precedenza sui posti riservati. Se però questi sono finiti, il sistema gli assegna comunque un posto standard, sempre che ce ne sia uno libero: è quello che chiamo *fallback*. Se sono esauriti anche i posti standard, allora l'accesso viene negato anche a lui.

In uscita l'unico che passa dalla fase di pagamento è l'utente STD; abbonati e disabili escono direttamente. Tutto questo comportamento l'ho prima descritto in modo formale con una macchina a stati (ASM) in Asmeta, poi l'ho validato con scenari di simulazione (Avalla) e con alcune proprietà di model checking (CTL), e infine l'ho implementato in Java mettendo i contratti JML sul nucleo del dominio.

1.2 Requisiti funzionali

ID	Descrizione	Priorità
RF1	Rilevare l'arrivo di un veicolo in ingresso (sens_in) e passare da IDLE a CHK_IN.	Alta
RF2	Identificare il tipo di utente (STD, DISABILE, ABBONATO) e passare allo stato VER.	Alta
RF3	Per STD/ABBONATO, assegnare un posto standard se posti_std > 0 e aprire la sbarra (INGR).	Alta
RF4	Per DISABILE, assegnare in via prioritaria un posto disabile se posti_dis > 0.	Alta
RF5	Se per un DISABILE i posti disabili sono esauriti, tentare il fallback su posto standard.	Alta
RF6	Se nessun posto è disponibile (inclusi i fallback), negare l'accesso (NEG).	Alta
RF7	Da NEG, tornare in IDLE solo dopo che il veicolo si è allontanato (auto_via).	Media
RF8	Al transito in ingresso, decrementare il contatore del posto assegnato e tornare in IDLE.	Alta
RF9	Rilevare l'arrivo di un veicolo in uscita (sens_out) e passare a CHK_OUT.	Alta
RF10	In uscita, per STD richiedere il pagamento (TARIF) prima della sbarra.	Alta
RF11	In uscita, per DISABILE/ABBONATO saltare il pagamento e andare direttamente a USC.	Alta
RF12	Restare in TARIF finché il pagamento non è confermato.	Alta
RF13	Al transito in uscita, incrementare il contatore, azzerare la memoria del posto e tornare in IDLE.	Alta

ID	Descrizione	Priorità
RF14	Mantenere in memoria il tipo di posto occupato dall'ingresso fino all'uscita.	Media

1.3 Requisiti non funzionali

ID	Descrizione
RNF1	Sicurezza dei contatori: <code>posti_std</code> e <code>posti_dis</code> non devono mai essere negativi né superare la capacità iniziale, in ogni stato raggiungibile (verificato via model checking).
RNF2	Determinismo: a parità di stato e input monitorati, il sistema produce sempre lo stesso stato successivo.
RNF3	Assenza di stati bloccanti: da ogni stato transitorio esiste sempre un cammino che riporta il sistema in IDLE.

1.4 Macchina a stati

Nella Figura 1 ho riportato gli 8 stati operativi del modello (`SmartParking.asm`) e le transizioni principali che li collegano.

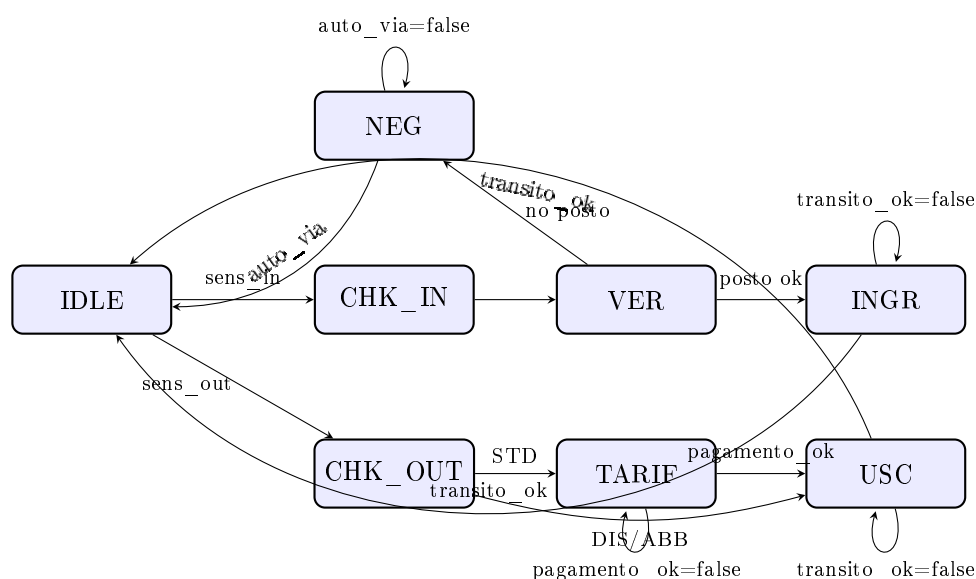


Figure 1: Diagramma della macchina a stati SmartParking (8 stati).

1.5 Glossario

Termine	Significato
Sensore ingresso/uscita	di Rileva la presenza di un veicolo davanti a una sbarra.
Sbarra	Barriera fisica aperta solo in INGR/USC quando <code>transito_ok=true</code> .

Termine	Significato
Transito	Il veicolo è passato fisicamente sotto la sbarra aperta.
Tipo utente	Categoria del veicolo: STD, DISABILE o ABBONATO.
Posto standard / disabile Fallback	Le due categorie di posto auto. Politica per cui un DISABILE senza posti riservati riceve un posto standard.
Tariffa (TARIF)	Fase di pagamento, obbligatoria solo per STD in uscita.

2 Modello Asmeta

2.1 Il modello (asmeta/src/SmartParking.asm)

Nel modello ho definito tre domini enumerativi (`StatoSistema`, `TipoUtente`, `TipoPosto`), sei funzioni monitorate (`sens_in`, `sens_out`, `transito_ok`, `auto_via`, `utente_rilevato`, `pagamento_ok`) e quattro funzioni controllate (`stato`, `posti_std`, `posti_dis`, `posto_assegnato`). A queste ho aggiunto una funzione derivata n-aria (binaria), `assegnabile: Prod(TipoUtente, TipoPosto) → Boolean`, che raccoglie in un punto solo tutta la politica di assegnazione: un `POSTO_STD` si può dare a chiunque se `posti_std > 0`, mentre un `POSTO_DIS` è riservato ai disabili e solo se `posti_dis > 0`. Trattandosi di una funzione derivata, viene ricalcolata a ogni stato a partire dalle funzioni controllate, quindi non occupa memoria.

Listing 1: Funzione derivata assegnabile

```

1 derived assegnabile: Prod(TipoUtente, TipoPosto) -> Boolean
2
3 function assegnabile($u in TipoUtente, $p in TipoPosto) =
4     ($p = POSTO_STD and posti_std > 0) or
5     ($p = POSTO_DIS and $u = DISABILE and posti_dis > 0)

```

Sempre tra le derivate ho aggiunto `parcheggioPieno`, che uso per far vedere il costrutto `forall`: è vera quando, per *ogni* tipo di utente, non è assegnabile né un posto standard né uno disabili, cioè quando davvero non può più entrare nessuno. La controllo nello scenario di saturazione `test_pieno_totale.avalla`, dove all'inizio è falsa (c'è ancora posto) e a parcheggio pieno diventa vera.

Listing 2: Funzione derivata `parcheggioPieno` (uso di `forall`)

```

1 function parcheggioPieno =
2     (forall $u in TipoUtente with
3         (not assegnabile($u, POSTO_STD) and not assegnabile($u, POSTO_DIS)))

```

La regola più importante è `r_gestione_VER`, quella che decide a chi assegnare il posto. Qui ho usato una regola `let` insieme alla derivata `assegnabile`: la regola prova prima con `POSTO_DIS` (che è vero solo per un disabili quando ci sono ancora posti disabili liberi), poi con `POSTO_STD` (il caso di STD e abbonati, ma anche il fallback del disabili quando i posti riservati sono finiti) e, se non va a buon fine nessuno dei due, nega l'accesso mandando il sistema in NEG:

Listing 3: Regola `r_gestione_VER` (assegnazione posto, con `let` e derivata)

```

1 rule r_gestione_VER =
2     let ($u = utente_rilevato) in
3         if assegnabile($u, POSTO_DIS) then
4             par

```

```

5      stato := INGR
6      posto_assegnato := POSTO_DIS
7  endpar
8  else
9      if assegnabile($u, POSTO_STD) then
10         par
11             stato := INGR
12             posto_assegnato := POSTO_STD
13         endpar
14     else
15         stato := NEGR
16     endif
17 endif
18 endlet

```

Per il model checking ho poi usato una versione semplificata del modello (SmartParking-MC.asm). Qui ho lasciato gli if annidati "appiattiti", che si comportano esattamente allo stesso modo ma rendono più facile la traduzione verso NuSMV. Sempre per lo stesso motivo, in questo modello i contatori non sono più Integer ma stanno su un dominio finito $\text{Posti} = \{0..2\}$, dato che AsmetaSMV non accetta domini infiniti. Ho scelto $\{0..2\}$ e non $\{0,1\}$ apposta: così le proprietà $\text{ag}(\text{posti_std} \leq 1)$ e $\text{ag}(\text{posti_dis} \leq 1)$ sono verifiche vere e proprie, e non diventano ovvie solo per come è fatto il tipo.

2.1.1 Simulazione manuale

Prima di lanciare gli scenari automatici, per prendere un po' di confidenza con il modello, conviene simulare a mano qualche step con il simulatore di Asmeta.

	Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5
 M	sens_in		false	true	true	true	true	
 M	sens_out		false	false	false	false	false	
 C	stato		IDLE	IDLE	CHK_IN VER	INGR	IDLE	
 M	utente_rilevato					STD	STD	
 C	posti_std					1	1	0
 C	posti_dis					1	1	1
 C	posto_assegnato						POSTO_STD	POSTO_STD
 M	transito_ok						true	

Figure 2: Simulatore – ingresso STD completo

2.2 Scenari Avalla

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10
M	sens_in	false	true	true	true	true	false	false	false	false	false	
M	sens_out	false	false	false	false	false	true	true	true	true	true	
C	stato	IDLE	IDLE	CHK_IN	VER	INGR	IDLE	CHK_OUT	TARIF	TARIF	USC	IDLE
M	utente_rilevato				STD	STD	STD	STD	STD	STD	STD	
C	posti_std				1	1	0	0	0	0	0	1
C	posti_dis				1	1	1	1	1	1	1	1
C	posto_assegnato					POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO
M	transito_ok					true	true	true	true	true	true	
M	pagamento_ok								false	true	true	

Figure 3: Simulatore – uscita STD con pagamento

Scenario	Obiettivo
test_abbonato.avalla	Ciclo completo ABBONATO: ingresso senza pagamento, uscita diretta (salta TARIF). Bug originale corretto: init <code>posti_std</code> non era 5 ma 1, e lo stato PARK non esiste (è IDLE).
test_disabile.avalla	Ciclo DISABLE: assegnazione POSTO_DIS, <code>posto_assegnato</code> resta POSTO_DIS durante la sosta, azzerato solo in uscita.
test_pieno.avalla	Un utente STD occupa l'unico posto; un secondo STD viene respinto (NEG).
test_disabile_- fallback.avalla	Copre il fallback: un secondo DISABLE, con <code>posti_dis=0</code> , riceve POSTO_STD.
test_pagamento_- std.avalla	Ciclo di uscita STD con un pagamento fallito (resta in TARIF) seguito da uno riuscito.
test_pieno_- totale.avalla	Parcheggio completamente saturo (STD + DISABLE), un terzo utente DISABLE viene respinto.
test_random_- 30steps.avalla	Scenario <i>generato automaticamente</i> : 30 step random eseguiti nel simulatore ed esportati con “export to avalla” (saturazione del parcheggio, doppio NEG, ciclo di pagamento TARIF, rientro).

Type	Functions	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13
C	step__	4	5	6	7	8	8	8	8	8	8
C	stato	IDLE	CHK_OUT	USC	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE	IDLE
C	sens_in	false	false	false	false	false	false	false	false	false	false
C	result	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato	ATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO	ABBONATO
C	posto_assegnato	STD	POSTO_STD	POSTO_STD	POSTO_STD	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO
C	transito_ok	false	false	true	true	true	true	true	true	true	true
C	posti_std	0	0	0	1	1	1	1	1	1	1
C	sens_out	true	false	false	false	false	false	false	false	false	false

Figure 4: Animazione di test_abbonato.avalla

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11
C	step__	0	1	2	3	4	5	6	7	8	9	9	9
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE
C	posto_assegnato				POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C	transito_ok				true	false	false	false	true	true	true	true	true
C	posti_dis				0	0	0	0	0	0	0	0	0
C	posti_std									0	0	0	0

Figure 5: Animazione di test_disabile_fallback.avalla

Type	Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C	step__	0	1	2	3	4	5	6	7	8	9	10	11	12	13	13	13
C	stato		CHK_IN	VER	INGR	IDLE	CHK_IN	VER	INGR	IDLE	CHK_IN	VER	NEG	IDLE	IDLE	IDLE	IDLE
C	sens_in		false	false	false	true	false	false	false	true	false	false	false	false	false	false	false
C	result		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
C	utente_rilevato		STD	STD	STD	STD	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	DISA
C	posto_assegnato				POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POSTO_DIS	POST
C	transito_ok				true	false	false	false	true	false	false	false	false	false	false	false	false
C	posti_std				0	0	0	0	0	0	0	0	0	0	0	0	0
C	posti_dis								0	0	0	0	0	0	0	0	0
C	auto_via											true	true	true	true	true	true

Figure 6: Animazione di test_pieno_totale.avalla

33%, $r_gestione_TARIF$ 50%, $r_gestione_IDLE$ 75%). Il ramo “pagamento fallito” di $TARIF$, per esempio, non capita quasi mai per caso, e infatti viene coperto solo dallo scenario mirato `test_pagamento_std`. Insomma, la generazione random è utile come aggiunta, ma non sostituisce gli scenari scritti a mano sui casi limite.

2.4 Model checking (asmeta/src/SmartParking_MC.asm)

Per il model checking ho scritto 10 CTLSPEC (Tabella 5). Otto me le aspetto vere e le ho divise in proprietà di sicurezza (safety), di raggiungibilità e di liveness; le altre due invece le ho messe apposta false, giusto per far vedere che il model checker, quando una proprietà non regge, tira fuori un controesempio.

#	CTLSPEC	Categoria	Esito
1	$ag(posti_std \geq 0)$	Safety	TRUE
2	$ag(posti_dis \geq 0)$	Safety	TRUE
3	$ag((stato=INGR \text{ and } posto_assegnato=POSTO_STD) \text{ implies } posti_std>0)$	Safety	TRUE
4	$ag(posti_std \leq 1)$	Safety	TRUE
5	$ag(posti_dis \leq 1)$	Safety	TRUE

#	CTLSPEC	Categoria	Esito
6	<code>ag((stato=INGR and posto_-assegnato=POSTO_DIS) implies posti_-dis>0)</code>	Safety	TRUE
7	<code>ef(stato = TARIF)</code>	Raggiungibilità	TRUE
8	<code>ag(stato=TARIF implies ef(stato=IDLE))</code>	Liveness	TRUE
9	<code>ag(stato != NEG)</code>	Falsa (voluta)	FALSE
10	<code>ag(stato=TARIF implies af(stato=IDLE))</code>	Falsa (voluta)	FALSE

Table 5: Le 10 CTLSPEC sottoposte ad AsmetaSMV: 8 verificate, 2 volutamente false.

Le due specifiche false sono interessanti, ognuna a modo suo:

- la #9 dice che “l’accesso non viene mai negato”. Il controesempio che genera NuSMV (9 stati) fa entrare un abbonato che occupa l’unico posto standard; poi arriva un secondo veicolo, la verifica non trova posto e il sistema va in NEG, e lo fa con il posto disabili ancora libero (`posti_dis = 1`). Questa traccia dimostra due cose in una volta: che lo stato di rifiuto si può davvero raggiungere, e che la riserva funziona, perché il posto disabili non viene dato a chi non ne ha diritto;
- la #10 è la versione con `af` della liveness #8: chiede che da TARIF il sistema torni in IDLE su *ogni* cammino, non solo che esista un cammino che ci riporta. È falsa perché nulla vieta all’ambiente di tenere `pagamento_ok = false` per sempre, cioè un utente che non paga mai, e infatti il controesempio è proprio il loop infinito in TARIF. Mettere a confronto la #8 e la #10 fa vedere bene la differenza tra `ef` e `af` in CTL.

2.4.1 Model Advisor (opzionale)

Il Model Advisor (AsmetaMA) lavora passando per la traduzione in NuSMV, quindi l’ho lanciato sul modello semplificato `SmartParking_MC.asm`: quello principale, con `Integer` e funzioni derivate, non è traducibile e dà “Error Domain Integer not supported”. L’analisi delle meta-proprietà ha dato questi esiti:

- **MP1, MP3, MP4, MP7** non hanno prodotto segnalazioni: nessun aggiornamento inconsistente, nessuna assegnazione banale, nessuna locazione rimuovibile o mai aggiornata.
- **MP2** (“conditional rule not complete”) scatta su tutte le regole di transizione, ma si tratta di falsi positivi noti. L’advisor segnala ogni `if` privo di `else`, mentre in ASM l’assenza dell’else è idiomatica e significa “nessun aggiornamento”: è proprio il comportamento che serve per i self-loop della FSM (per esempio `r_gestione_NEG` resta in NEG finché `auto_via` non diventa vero, e `r_gestione_TARIF` resta in TARIF a pagamento fallito).
- **MP5/MP6** (“Domain Posti could be reduced: element {2} could be removed”) non segnala un difetto ma conferma una scelta. Il dominio `Posti = {0..2}` è stato preso apposta più ampio del necessario, e l’advisor dimostra che il valore 2 non è mai raggiunto da `posti_std/posti_dis`, cioè la proprietà di capacità `ag(posti <= 1)` verificata anche dalle CTLSPEC.

```

** Coverage Info: **
** covered rules: **
SmartParking::r_gestione_USC()
-> branch coverage:      33.33%
-> rule coverage:        75.00%
-> update rule coverage: 75.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_IN()
-> branch coverage:      - (no branches to be covered)
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_INGR()
-> branch coverage:      33.33%
-> rule coverage:        71.43%
-> update rule coverage: 66.67%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_VER()
-> branch coverage:      50.00%
-> rule coverage:        60.00%
-> update rule coverage: 40.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_IDLE()
-> branch coverage:      75.00%
-> rule coverage:        100.00%
-> update rule coverage: 100.00%
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_Main()
-> branch coverage:      87.50%
-> rule coverage:        88.24%
-> update rule coverage: - (no update rules to be covered)
-> forall rule coverage: - (no forall rules to be covered)
SmartParking::r_gestione_CHK_OUT()
-> branch coverage:      50.00%
-> rule coverage:        66.67%
-> update rule coverage: 50.00%
-> forall rule coverage: - (no forall rules to be covered)
** NOT covered rules: **
SmartParking::r_gestione_NEG()
SmartParking::r_gestione_TARIF()

```

Figure 7: Report di copertura AsmetaV (Vc)

2.4.2 ATGT – generazione automatica di scenari (opzionale)

Gli scenari che ATGT genera (`asmeta/src/abstracttests/testtest*.avalla`) si possono guardare direttamente nel repository. Il loro ruolo, insieme a quello dello scenario random da 30 step, l'ho già discusso nel confronto con gli scenari scritti a mano in §2.3.

3 Implementazione Java

3.1 Struttura del progetto Maven

Il progetto Maven (`java/`) gira su Java 17 e usa Spring Boot con Vaadin 24 per la UI, JUnit 5 e Mockito per i test e JaCoCo per la copertura. Il cuore del programma sta nel package `it.tvsw.smartparking.core`, che contiene:

```

model checking (w execution) on /Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/eclipse_asmeta_
Execution of NuSMV code ...
-----
> NuSMV -dynamic -coi -quiet SmartParking_MC.smv
-- specification AG posti_std >= 0 is true
-- specification AG posti_dis >= 0 is true
-- specification AG ((posto_assegnato = POSTO_STD & stato = INGR) -> posti_std > 0) is true
-- specification AG posti_std <= 1 is true
-- specification AG posti_dis <= 1 is true
-- specification AG ((stato = INGR & posto_assegnato = POSTO_DIS) -> posti_dis > 0) is true
-- specification EF stato = TARIF is true
-- specification AG (stato = TARIF -> EF stato = IDLE) is true
-- specification AG stato != NEG is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 1.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 1.2 <-
  sens_in = true
-> State: 1.3 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.4 <-
  stato = VER
-> State: 1.5 <-
  stato = INGR
  transito_ok = true
  posto_assegnato = POSTO_STD
-> State: 1.6 <-
  stato = IDLE
  transito_ok = false
  sens_in = true
  posti_std = 0
-> State: 1.7 <-
  stato = CHK_IN
  sens_in = false
-> State: 1.8 <-
  stato = VER
-> State: 1.9 <-
  stato = NEG
-- specification AG (stato = TARIF -> AF stato = IDLE) is false
-- as demonstrated by the following execution sequence
Trace Description: CTL Counterexample
Trace Type: Counterexample
-> State: 2.1 <-
  stato = IDLE
  transito_ok = false
  auto_via = false
  sens_out = false
  sens_in = false
  utente_rilevato = ABBONATO
  pagamento_ok = false
  posti_dis = 1
  posti_std = 1
  posto_assegnato = NESSUNO
-> State: 2.2 <-
  sens_out = true
-> State: 2.3 <-
  stato = CHK_OUT
  sens_out = false
  utente_rilevato = STD
-- Loop starts here
-> State: 2.4 <-
  stato = TARIF
  utente_rilevato = ABBONATO
-> State: 2.5 <-

model checking (w execution) finished

```

Figure 8: Esito delle 10 CTLSPEC in AsmetaSMV

- StatoSistema, TipoUtente, TipoPosto: enum identici ai domini ASM;
- Parcheggio: nucleo con contratti JML (Sezione 5);

Type/Functions	State 0	State 1	State 2	State 3	State 4	State 5	State 6	State 7	State 8	State 9	State 10	State 11	State 12	State 13	State 14	State 15
C stato	IDLE	CHK_OUT	USC	USC	IDLE	CHK_OUT	USC	USC	IDLE	CHK_IN	VER	INGR	INGR	IDLE	CHK_IN	VER
M sens_in	false	false	false	false	false	false	false	false	true	true	true	true	true	true	true	
M sens_out	true	true	true	true	true	true	true	true	true	true	true	true	true	true	true	
M utente_rilevato		ABBONATO	ABBONATO	ABBONATO	ABBONATO	DISABILE	DISABILE	DISABILE	DISABILE	DISABILE	STD	STD	STD	STD	STD	
M transito_ok			false	true	true	true	false	true	true	true	true	false	true	true	true	
C posto_assegnato				NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	NESSUNO	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD	POSTO_STD
C posti_std											1	1	1	0	0	0
C posti_dis											1	1	1	1	1	1

Figure 9: Esecuzione random-step di verifica manuale

```

MP2: Every case rule without otherwise must be complete
NONE

MP3: Choose rule is always/sometimes/never not empty
NONE

MP3: Forall rule is always/sometimes/never not empty
NONE

MP3: Conditional rule eval to true
NONE

MP4: No assignment is always trivial
NONE

MP5: For every domain element e, there exists a location which has value e
Domain Posti could be reduced in size. Elements {2} could be removed.

MP6: Every controlled location can take any value in its codomain
Function posti_dis does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.
Function posti_std does not take the values {2} of its domain. It could be defined over the smaller domain {0, 1}.

MP7: a location could be removed
NONE

MP7: a controlled location is never updated
NONE

MP7: a controlled location could be static
NONE

model advising finished

```

Figure 10: Output del Model Advisor (estratto)

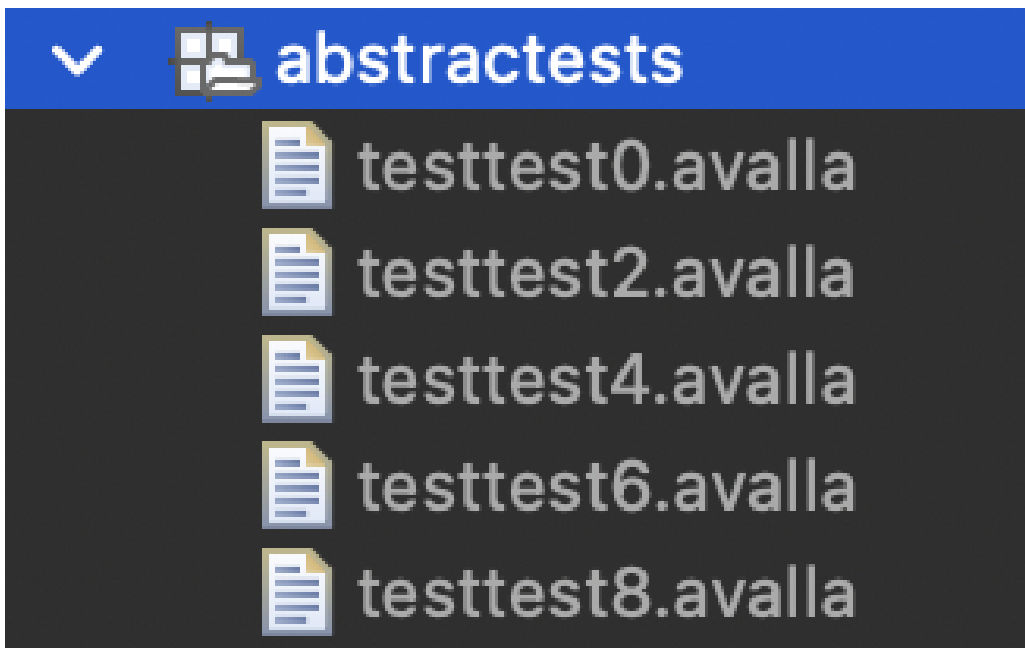


Figure 11: Elenco degli scenari generati da ATGT

- **SensorInput**: raccoglie i 6 input monitorati di uno step;
- **Display**: interfaccia di notifica (usata per Mockito);
- **SmartParkingFSM**: la macchina a stati, un metodo `step(SensorInput)` che replica la main rule ASM.

3.2 La macchina a stati

Listing 4: `step()` – dispatch sullo stato corrente

```

1 public void step(SensorInput input) {
2     switch (stato) {
3         case IDLE:      gestisciIdle(input); break;
4         case CHK_IN:    gestisciChkIn(); break;
5         case VER:       gestisciVer(input); break;
6         case NEG:       gestisciNeg(input); break;
7         case INGR:      gestisciIngr(input); break;
8         case CHK_OUT:   gestisciChkOut(input); break;
9         case TARIF:     gestisciTarif(input); break;
10        case USC:       gestisciUsc(input); break;
11    }
12 }
```

Una nota sul design: nel modello ASM il contatore viene decrementato solo al transito vero e proprio in INGR, mentre qui la classe **Parcheggio** decide e decrementa tutto in un colpo solo, dentro **assegnaPosto** (che viene chiamato da **gestisciVer**). Negli scenari di test del progetto questa differenza non si vede, e l’ho scelta apposta per tenere il codice più semplice; ne riparlo meglio nell’osservazione dedicata in Sezione 6.

3.3 Interfaccia utente (Vaadin)

Per la UI ho creato la vista **MainView**, che mostra lo stato corrente e i posti ancora liberi. I sensori li ho simulati con dei bottoni: “Arriva auto STD/DISABILE/ABBONATO”, “Transito”, “Pagamento”, “Auto va via” e “Auto in uscita”. In pratica cliccando i bottoni si fa avanzare la macchina a stati come se arrivassero gli input dai sensori veri.

3.4 Test end-to-end con Selenium

La UI l’ho controllata anche con un test end-to-end fatto in Selenium (**UISeleniumTest**). L’ho taggato “ui” e l’ho escluso dalla build normale, perché per girare ha bisogno dell’applicazione già avviata e di Chrome. Quello che fa è aprire la pagina con **ChromeDriver** in modalità headless, aspettare che Vaadin finisca di renderizzare lato client (con **WebDriverWait**), cliccare “Arriva auto STD” e controllare che lo stato mostrato diventi INGR o NEG. Per lanciarlo basta `mvn test -Dtest.groups=ui -Dtest.excludedGroups=`, oppure eseguirlo da Eclipse come un normale test JUnit.

4 Testing Java

4.1 Struttura dei test

Ho diviso i test in più classi, una per ogni aspetto che volevo coprire, così è più facile capire cosa controlla cosa. Nella tabella qui sotto ho messo l’elenco con, accanto, quello che ciascuna classe va a verificare.

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 1

Posti disabili liberi: 1

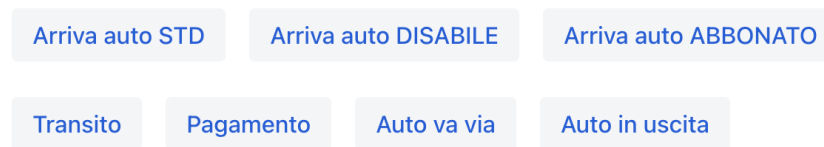


Figure 12: UI Vaadin – stato iniziale

SmartParking - Simulatore

Stato: IDLE

Posti standard liberi: 0

Posti disabili liberi: 1

Sistema pronto

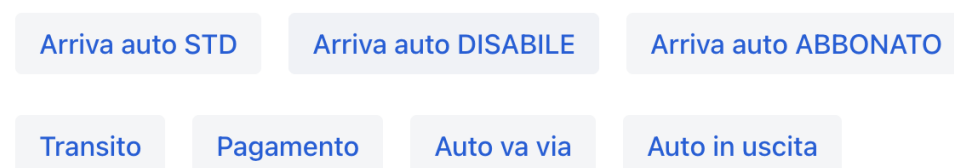


Figure 13: UI Vaadin – dopo un ingresso STD

Classe	Copre
ParcheggioTest	Costruttori, preconditioni violate, tutti i rami di assegnaPosto/liberaPosto , test parametrico CSV.
SmartParkingFSMTest	Tutti gli 8 stati e le transizioni, fallback disabile, parcheggio pieno, self-loop NEG/INGR/TARIF.

Classe	Copre
McdcVerificaTest	Copertura MC/DC delle due decisioni composte di assegnaPosto (Tabella 7 e 8).
DisplayMockTest	Notifiche a Display verificate con Mockito.
ScenarioAvallaTest	Traduzione fedele di test_pieno.avalla in JUnit (Model Based Testing).
CombinatorialTest	Tabella pairwise a 6 righe (Tabella 9).
UISeleniumTest	Smoke test della UI (tag ui , escluso dalla build standard).

4.2 MC/DC

Dentro **assegnaPosto** ci sono due decisioni composte, cioè con più condizioni messe in and/or, e per queste ho voluto una copertura MC/DC vera e propria: per ogni condizione serve una coppia di casi di test in cui, cambiando solo quella condizione, cambia anche il risultato della decisione. Vediamole una alla volta.

La prima decisione è quella del ramo STD/ABBONATO: $D_1 = (\text{utente}=\text{STD} \vee \text{utente}=\text{ABBONATO}) \wedge \text{postiStd}>0$.

Caso	A	B	C	D_1	Esito	Coppia indipendente
TC1	T	F	T	T	POSTO_STD	base
TC2	T	F	F	F	NESSUNO	isola C (vs TC1)
TC3	F	F	T	F	POSTO_DIS	isola A (vs TC1)
TC4	F	T	T	T	POSTO_STD	isola B (vs TC3)

Table 7: MC/DC – Decisione 1.

La seconda decisione è quella del ramo DISABILE, cioè il fallback, con $C_1=\text{postiDis}>0$ e $C_2=\text{postiStd}>0$.

Caso	C_1	C_2	Esito	Coppia indipendente
TC5	T	—	POSTO_DIS	base
TC6	F	T	POSTO_STD	isola C_1 (vs TC5)
TC7	F	F	NESSUNO	isola C_2 (vs TC6)

Table 8: MC/DC – Decisione 2.

4.3 Combinatorial testing (pairwise)

L'assegnazione del posto dipende da tre parametri: il tipo di utente ($\text{tipoUtente} \in \{\text{STD}, \text{DISABILE}, \text{ABBONATO}\}$), i posti standard liberi ($\text{postiStd} \in \{0, 1\}$) e quelli disabili liberi ($\text{postiDis} \in \{0, 1\}$). Provare tutte le combinazioni sarebbe possibile ma inutile: mi basta coprire tutte le *coppie* di valori, ed è quello che fa la tabella pairwise qui sotto, che con sole 6 righe le copre tutte.

#	tipoUtente	postiStd	postiDis	Esito atteso
1	STD	0	0	NESSUNO
2	STD	1	1	POSTO_STD
3	DISABILE	0	1	POSTO_DIS

#	tipoUtente	postiStd	postiDis	Esito atteso
4	DISABILE	1	0	POSTO_STD (fallback)
5	ABBONATO	0	1	NESSUNO
6	ABBONATO	1	0	POSTO_STD

Table 9: Tabella pairwise (copre tutte le coppie dei 3 parametri).

4.4 Mockito

La FSM, a ogni cambio di stato, manda un messaggio a un oggetto **Display**. Per controllare che questi messaggi partano davvero, senza dover tirare in ballo la UI vera, ho usato Mockito: creo un finto **Display** e poi con **verify** controllo che il metodo **mostra** venga chiamato con il testo giusto. Nell'esempio qui sotto riempio il parcheggio e verifico che, al terzo ingresso, il display riceva davvero "Accesso negato".

Listing 5: Verifica delle notifiche con Mockito

```

1 @Test
2 void notificaAccessoNegatoQuandoParcheggioPieno() {
3     SmartParkingFSM fsm = new SmartParkingFSM(new Parcheggio(0, 0), display);
4     fsm.step(SensorInput.sensIn());
5     fsm.step(SensorInput.utente(TipoUtente.STD));
6     fsm.step(SensorInput.utente(TipoUtente.STD));
7     verify(display).mostra("Accesso negato");
8 }

```

4.5 Model Based Testing

L'idea del model based testing è prendere uno scenario già scritto sul modello astratto e tradurlo in un test sul codice vero. Ho preso lo scenario **test_pieno.avalla** e l'ho riportato passo passo in **ScenarioAvallaTest**: ogni blocco **set/step/check** dell'avalla diventa la corrispondente riga JUnit, e ho lasciato un commento accanto a ciascuna per richiamare la riga originale dello scenario.

4.6 Esecuzione e copertura

La copertura l'ho misurata con JaCoCo, lanciando **mvn verify** e guardando il report in **target/site/jacoco/in**. Dal conteggio ho tolto la UI Vaadin (che testo end-to-end con Selenium e che JaCoCo non riesce a misurare) e la classe di avvio di Spring Boot, che sono solo 3 righe di boilerplate; l'esclusione è scritta nel **pom.xml**. Alla prima misurazione il package **core** stava al 98% di instruction e al 95% di branch. La classe **Parcheggio** era già al 100%/100% (coerente con la verifica ESC dei contratti, §5), mentre in **SmartParkingFSM** restavano scoperti 3 branch: il self-loop di **gestisciUsc** (**transito_ok** falso in USC), il ramo **display == null** di **cambiaStato** e il default dello switch di **step**. I primi due erano veri buchi di test, che ho chiuso aggiungendo due test mirati (**uscRestaInUscFinoAlTransito**, **fsmFunzionaSenzaDisplay**). Il terzo invece non è raggiungibile: lo switch gestisce tutti e 8 i valori dell'enum **StatoSistema**, per cui il default (difensivo, lancia **IllegalStateException**) non può mai essere eseguito. È l'analogo Java dei rami infattibili già osservati con AsmetaV sul modello (§2.3). Dopo aver aggiunto i due test, quel default resta l'unico branch non coperto del package.

SmartParking - Simulatore

Stato: NEG

Posti standard liberi: 0

Posti disabili liberi: 1

Accesso negato

Arriva auto STD

Arriva auto DISABILE

Arriva auto ABBONATO

Transito

Pagamento

Auto va via

Auto in uscita

Figure 14: UI Vaadin – accesso negato a parcheggio pieno

SmartParking - Simulatore

Stato: TARIF

Posti standard liberi: 0

Posti disabili liberi: 1

In attesa di pagamento

Arriva auto STD

Arriva auto DISABILE

Arriva auto ABBONATO

Transito

Pagamento

Auto va via

Auto in uscita

Figure 15: UI Vaadin – attesa pagamento

5 JML e verifica statica (ESC)

Per la parte di design by contract ho annotato con JML la classe `Parcheggio` (package `it.tvsw.smartparking.c`) che è il nucleo puro del dominio: tiene i contatori dei posti liberi e gestisce l'assegnazione e il rilascio, cioè la stessa logica delle regole `r_gestione_VER`, `r_gestione_INGR` e `r_gestione_USC`

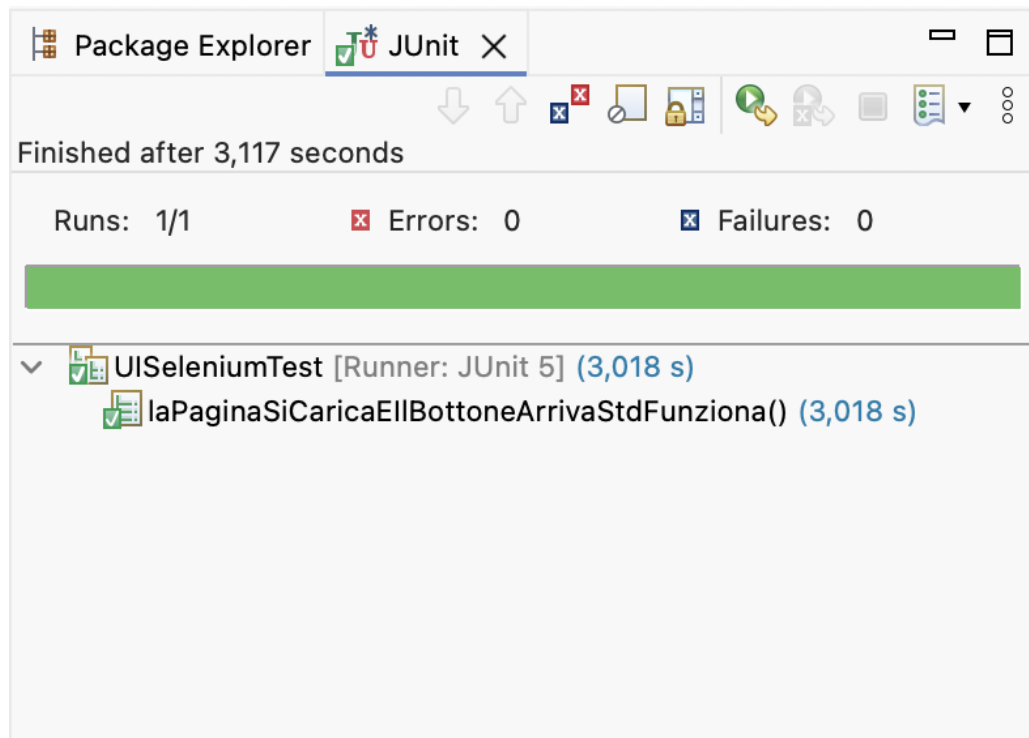


Figure 16: Test Selenium end-to-end superato

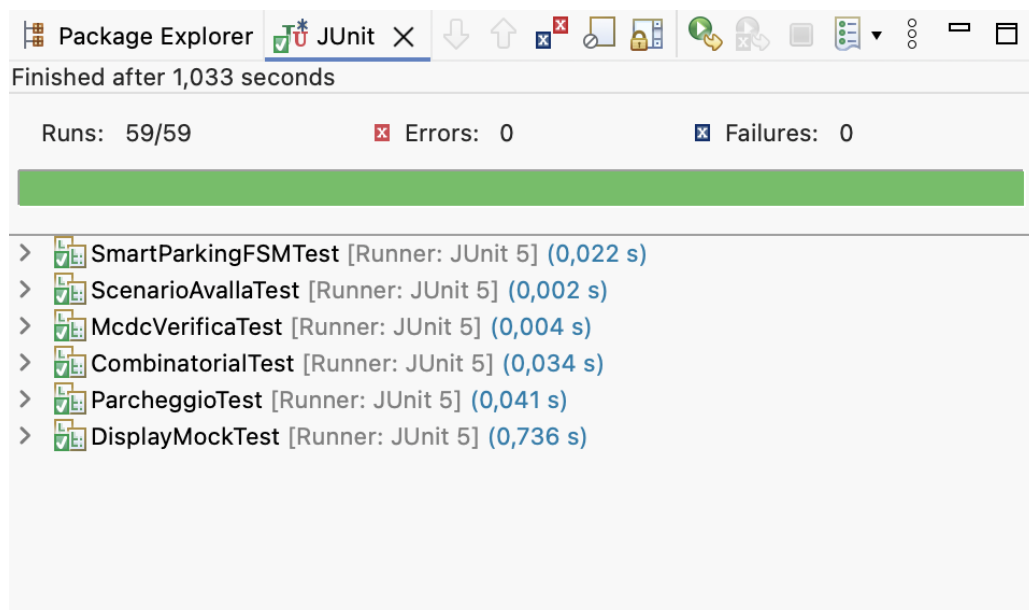
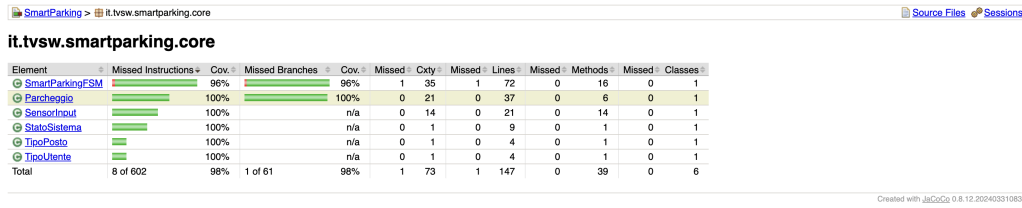


Figure 17: Esecuzione di tutta la suite JUnit

SmartParking										
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cnty	Missed Lines	Missed Methods	Missed Classes		
it.tvsw.smartparking.core	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39
Total	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39

Figure 18: Report JaCoCo – totale (solo package core)

del modello ASM. L'ho tenuta piccola apposta e senza dipendenze esterne (dentro ci sono solo



SmartParking > it.tvsw.smartparking.core

Source Files Sessions

it.tvsw.smartparking.core

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
SmartParkingFSM	98%	98%	98%	98%	1	35	1	72	0	16	0	1
Parcheggio	100%	100%	100%	100%	0	21	0	37	0	6	0	1
SensorInput	100%	100%	n/a	n/a	0	14	0	21	0	14	0	1
StatoSistema	100%	100%	n/a	n/a	0	1	0	9	0	1	0	1
TipoPosto	100%	100%	n/a	n/a	0	1	0	4	0	1	0	1
TipoUtente	100%	100%	n/a	n/a	0	1	0	4	0	1	0	1
Total	8 of 602	98%	1 of 61	98%	1	73	1	147	0	39	0	6

Created with JaCoCo 0.8.12.202403310830

Figure 19: Report JaCoCo – dettaglio per classe del package core

SmartParking > it.tvsw.smartparking.core > SmartParkingFSM

Sessions

SmartParkingFSM

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
step(SensorInput)	84%	90%	1	10	1	21	0	1		
SmartParkingFSM(Parcheggio, Display)	100%	100%	0	2	0	8	0	1		
gestisciVeri(SensorInput)	100%	100%	0	2	0	7	0	1		
gestisciIdler(SensorInput)	100%	100%	0	3	0	5	0	1		
gestisciUsco(SensorInput)	100%	100%	0	2	0	5	0	1		
gestisciChioOut(SensorInput)	100%	100%	0	2	0	5	0	1		
cambiaStato(SensorInput, String)	100%	100%	0	2	0	4	0	1		
gestisciNegi(SensorInput)	100%	100%	0	2	0	3	0	1		
gestisciIngi(SensorInput)	100%	100%	0	2	0	3	0	1		
gestisciTariff(SensorInput)	100%	100%	0	2	0	3	0	1		
SmartParkingFSM(Display)	100%	n/a	0	1	0	2	0	1		
gestisciChioIn()	100%	n/a	0	1	0	2	0	1		
getPostiStd()	100%	n/a	0	1	0	1	0	1		
getPostiDis()	100%	n/a	0	1	0	1	0	1		
getStato()	100%	n/a	0	1	0	1	0	1		
getPostoAssegnato()	100%	n/a	0	1	0	1	0	1		
Total	8 of 204	96%	1 of 31	96%	1	35	1	72	0	16

Created with JaCoCo 0.8.12.202403310830

Figure 20: Report JaCoCo – dettaglio per metodo di SmartParkingFSM

int ed enum), proprio perché così OpenJML riesce a dimostrarla tutta in modalità Extended Static Checking. Come chiedeva la consegna, il resto del sistema (la FSM, la UI Vaadin, il display, i sensori) è Java normale, senza nessuna annotazione JML.

Un dettaglio sulla visibilità: i campi `postiStd` e `postiDis` li ho dichiarati `private /*@ spec_public @*/`. Il motivo è che le clausole di specifica `public`, come le invarianti e le postcondizioni, possono nominare solo cose visibili a livello di specifica `public`; senza lo `spec_public` OpenJML si lamenta con un errore di visibilità.

5.1 Invarianti

Ho scritto due invarianti di classe, che devono valere in ogni stato osservabile dell'oggetto, cioè da dopo il costruttore in poi:

- il numero di posti standard liberi è sempre compreso tra 0 e la capacità massima `MAX_STD`;
- il numero di posti disabili liberi è sempre compreso tra 0 e la capacità massima `MAX_DIS`.

In pratica sono la versione Java delle proprietà di safety che avevo già verificato con il model checking sul modello ASM (`ag(posti >= 0)` e `ag(posti <= 1)`, si veda §2.3). Così la stessa proprietà finisce per essere garantita su tre livelli: il modello, il contratto e i test.

Listing 6: Invarianti di classe

```
//@ public invariant 0 <= postiStd && postiStd <= MAX_STD;
private /*@ spec_public @*/ int postiStd;

//@ public invariant 0 <= postiDis && postiDis <= MAX_DIS;
private /*@ spec_public @*/ int postiDis;
```

5.2 Costruttori

Per i costruttori ho scritto questi contratti:

- il costruttore di default garantisce (**ensures**) lo stato iniziale del modello ASM: tutti i posti liberi (`postiStd == MAX_STD, postiDis == MAX_DIS`);
- il costruttore con parametri (usato nei test combinatori e MC/DC per costruire configurazioni arbitrarie) *richiede* (**requires**) che i valori iniziali siano già dentro i limiti dell'invariante, e garantisce che i campi vengano inizializzati esattamente con i valori passati;
- la violazione della preconditione è anche gestita difensivamente a runtime con `IllegalArgumentException`, comportamento verificato in `ParcheggioTest` con `assertThrows`.

Listing 7: Contratti dei costruttori

```
//@ ensures postiStd == MAX_STD && postiDis == MAX_DIS;
public Parcheggio() { ... }

//@ requires 0 <= postiStdIniziali && postiStdIniziali <= MAX_STD;
//@ requires 0 <= postiDisIniziali && postiDisIniziali <= MAX_DIS;
//@ ensures postiStd == postiStdIniziali && postiDis == postiDisIniziali;
public Parcheggio(int postiStdIniziali, int postiDisIniziali) { ... }
```

5.3 Metodo assegnaPosto

Questo è il metodo con il contratto più corposo, perché deve rispecchiare tutta la logica di assegnazione di `r_gestione_VER`. I contratti sono questi:

- viene richiesto che l'utente non sia `null`;
- per gli utenti STD e ABBONATO la postcondizione descrive i due esiti possibili in funzione dello stato *precedente* (`\old`): se c'era un posto standard libero il risultato è `POSTO_STD` e il contatore è decrementato di 1; altrimenti il risultato è `NESSUNO` e nulla cambia;
- per gli utenti DISABILE la postcondizione descrive i tre esiti possibili: posto disabled se disponibile; altrimenti *fallback* su un posto standard se disponibile; altrimenti `NESSUNO` senza modifiche;
- in ogni esito viene anche vincolato che il contatore “dell'altro” tipo di posto resti invariato: la postcondizione è cioè *completa*, non lascia comportamenti sottospecificati.

Listing 8: Contratto di assegnaPosto

```
//@ requires utente != null;
//@ ensures (utente == TipoUtente.STD || utente == TipoUtente.ABBONATO) ==>
//@      ((\old(postiStd) > 0 && \result == TipoPosto.POSTO_STD
//@      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
//@      || (\old(postiStd) == 0 && \result == TipoPosto.NESSUNO
//@      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
//@ ensures utente == TipoUtente.DISABILE ==>
//@      ((\old(postiDis) > 0 && \result == TipoPosto.POSTO_DIS
//@      && postiDis == \old(postiDis) - 1 && postiStd == \old(postiStd))
//@      || (\old(postiDis) == 0 && \old(postiStd) > 0
//@      && \result == TipoPosto.POSTO_STD
//@      && postiStd == \old(postiStd) - 1 && postiDis == \old(postiDis))
//@      || (\old(postiDis) == 0 && \old(postiStd) == 0
//@      && \result == TipoPosto.NESSUNO
//@      && postiStd == \old(postiStd) && postiDis == \old(postiDis)));
public TipoPosto assegnaPosto(TipoUtente utente) { ... }
```

5.4 Metodo liberaPosto

Questo metodo corrisponde al rilascio del posto della regola `r_gestione_USC`. I contratti sono questi:

- viene richiesto che il posto non sia `null`;
- viene richiesto che il contatore corrispondente al posto da liberare non sia già al massimo: senza questa preconditione l'incremento violerebbe l'invariante superiore (non si possono "inventare" posti oltre la capacità);
- le postcondizioni garantiscono l'incremento di 1 del solo contatore corrispondente, e che con `NESSUNO` lo stato resti del tutto invariato.

Listing 9: Contratto di liberaPosto

```
//@ requires posto != null;
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
//@ ensures posto == TipoPosto.POSTO_STD ==>
//@   postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
//@ ensures posto == TipoPosto.POSTO_DIS ==>
//@   postiDis == \old(postiDis) + 1 && postiStd == \old(postiStd);
//@ ensures posto == TipoPosto.NESSUNO ==>
//@   postiStd == \old(postiStd) && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

5.5 Metodi getter

I due getter li ho annotati `/*@ pure @*/`, cioè senza effetti collaterali: è la condizione che serve per poterli usare dentro le clausole JML. La postcondizione dice solo che restituiscono il valore del campo, niente di più.

Listing 10: Getter puri

```
//@ ensures \result == postiStd;
public /*@ pure @*/ int getPostiStd() { ... }

//@ ensures \result == postiDis;
public /*@ pure @*/ int getPostiDis() { ... }
```

5.6 Verifica dei contratti con OpenJML (ESC)

Una volta scritti i contratti li ho verificati staticamente con OpenJML (release 21.0.27, build nativa per macOS arm64) in modalità `-esc`. Per ogni metodo il tool traduce codice e contratti in formule logiche e chiede al solver Z3 di dimostrare che *ogni* esecuzione possibile rispetta postcondizioni e invarianti; è un controllo più forte del testing, che invece guarda solo alcuni casi. Il comando, dalla root del progetto, è questo:

Listing 11: Comando di verifica ESC

```
$ openjml --esc --progress \
  -cp java/src/main/java \
  java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
```

Un contratto che inizialmente non chiudeva

Arrivare a “tutti Verified” non è stato immediato. Alcuni contratti, nella loro prima stesura, venivano rifiutati dal solver, e vale la pena mostrare perché e come sono stati corretti: è lo stesso tipo di ostacolo che rende difficile la chiusura ESC su codice con aritmetica non vincolata.

Il caso più chiaro è `liberaPosto`. Nella versione iniziale la postcondizione descriveva l’incremento del contatore, ma *manca la preconditione* che vieta di liberare un posto quando il contatore è già al massimo:

Listing 12: Prima versione di `liberaPosto` (senza il `requires` di capacità)

```
//@ requires posto != null;
//@ ensures posto == TipoPosto.POSTO_STD ==>
//@     postiStd == \old(postiStd) + 1 && postiDis == \old(postiDis);
public void liberaPosto(TipoPosto posto) { ... }
```

Con questo contratto, OpenJML esplora anche lo stato `postiStd == MAX_STD`: in quel caso `postiStd + 1` vale `MAX_STD + 1`, che *viola l’invariante di classe* `postiStd <= MAX_STD`. Il solver non riesce quindi a ristabilire l’invariante all’uscita del metodo e segnala l’errore:

Listing 13: Errore ESC sulla prima versione

```
Parcheggio.java:.. warning: The prover cannot establish an assertion
(InvariantExit) in method liberaPosto
  postiStd == \old(postiStd) + 1 ...
Associated declaration: invariant 0 <= postiStd && postiStd <= MAX_STD
```

È lo stesso ostacolo (un’operazione aritmetica che può “sforare” un limite) per cui su un dominio più ricco, con array e contatori non limitati, la chiusura completa diventa molto difficile. Qui la soluzione è rafforzare la preconditione, in modo che il chiamante garantisca lo spazio per l’incremento:

Listing 14: Versione corretta: il `requires` di capacità chiude la prova

```
//@ requires posto == TipoPosto.POSTO_STD ==> postiStd < MAX_STD;
//@ requires posto == TipoPosto.POSTO_DIS ==> postiDis < MAX_DIS;
```

Con `postiStd < MAX_STD` assunto all’ingresso, dopo l’incremento vale `postiStd <= MAX_STD` e l’invariante è ristabilita: il metodo passa a *Verified*. È proprio questa disciplina, cioè vincolare via `requires` o invariante tutto ciò che entra in un’operazione aritmetica, che permette a `Parcheggio` di chiudere tutti i contratti, cosa che su un nucleo con interi liberi non sarebbe possibile. Un secondo ostacolo, di natura diversa, era di pura visibilità JML: i campi `private` referenziati da clausole `public`. L’ho risolto con `spec_public`, come descritto nella nota a inizio sezione.

6 Analisi statica e ispezione del codice con LLM

Visto che la consegna chiedeva di fare l’ispezione “con chatgpt o LLM in generale”, ho deciso di usare ChatGPT come se fosse un collega a cui far dare una seconda occhiata al codice. Gli ho passato tutto il sorgente del progetto (i package `core` e `ui`) e gli ho chiesto di leggerlo riga per riga cercando nomi poco chiari, casi limite non gestiti, possibili `NullPointerException`, punti in cui il codice si allontana dal modello ASM e in generale debolezze di design. Le cose che sono uscite però non le ho prese per buone così com’erano: le ho valutate una per una io, decidendo di volta in volta se correggerle, mitigarle o lasciarle come stavano spiegando il perché. In pratica ho usato l’LLM per farmi notare i punti su cui ragionare, ma le decisioni le ho prese io, come in una code review vera.

```

/Users/francesco/Documents/INGEGNERIA/MAGISTRALE/TESTING/openjml-macos-arm64-21.0.27/openjml \
--esc --progress \
-cp java/src/main/java/it/tvsw/smartparking/core/Parcheggio.java
Proving methods in it.tvsw.smartparking.core.Parcheggio
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X
Method assertions are validated [1,14 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio() with prover z3-4.3.X - no warnings [1,26 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X
Method assertions are validated [1,21 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.Parcheggio(int,int) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X
Method assertions are validated [1,23 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.assegnaPosto(it.tvsw.smartparking.core.TipoUtente) with prover z3-4.3.X - no warnings [1,30 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X
Method assertions are validated [1,09 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiDis() with prover z3-4.3.X
Starting proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X
Method assertions are validated [1,05 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.getPostiStd() with prover z3-4.3.X - no warnings [1,05 secs]
Starting proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X
Method assertions are validated [1,19 secs]
Completed proof of it.tvsw.smartparking.core.Parcheggio.liberaPosto(it.tvsw.smartparking.core.TipoPosto) with prover z3-4.3.X - no warnings [1,22 secs]
Completed proving methods in it.tvsw.smartparking.core.Parcheggio [7,18 secs]
Proving methods in it.tvsw.smartparking.core.TipoUtente
Starting proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X
Method assertions are validated [1,06 secs]
Completed proof of it.tvsw.smartparking.core.TipoUtente.TipoUtente() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoUtente [1,07 secs]
Proving methods in it.tvsw.smartparking.core.TipoPosto
Starting proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X
Method assertions are validated [1,06 secs]
Completed proof of it.tvsw.smartparking.core.TipoPosto.TipoPosto() with prover z3-4.3.X - no warnings [1,07 secs]
Completed proving methods in it.tvsw.smartparking.core.TipoPosto [1,07 secs]
Summary:
Valid:      8
Invalid:    0
Infeasible: 0
Timeout:    0
Error:      0
Skipped:    0
TOTAL METHODS: 8
Classes:    3 proved of 3
DURATION:   9,6 secs

(base) francesco@iPhone Progetto_Testing %

```

Figure 21: Output di openjml -esc: tutti i metodi Verified

6.1 Checklist preliminare

- ✓ La funzionalità descritta nei requisiti è pienamente implementata dal codice.
- ✓ Tutte le variabili sono inizializzate prima dell'uso (i costruttori di `Parcheggio` validano i parametri).
- ✓ Gli enum (`TipoUtente`, `TipoPosto`, `StatoSistema`) sono confrontati con `==`, corretto per gli enum Java.
- ✓ Lo `switch` in `SmartParkingFSM.step` ha un ramo `default` che lancia eccezione su stato non gestito.
- ✓ Le eccezioni di preconditione (`IllegalArgumentException`, `IllegalStateException`) sono usate coerentemente per segnalare violazioni di contratto.

6.2 Osservazioni emerse e come sono state gestite

1. **Validazione mancante di `utenteRilevato==null` in `CHK_OUT`:** il confronto restituisce silenziosamente `false`, portando il sistema in USC come se l'utente non fosse STD. *Gestione:* accettata per fedeltà all'ASM (il dominio `TipoUtente` non modella l'assenza di dato); mitigata dal fatto che tutti i punti di chiamata impostano sempre l'utente.
2. **`TipoPosto.NESSUNO` con doppio significato** ("nessun posto disponibile" in `assegnaPosto` / "no-op" in `liberaPosto`). *Gestione:* chiarita via Javadoc, invece di introdurre un tipo separato che avrebbe appesantito una classe pensata per restare semplice.
3. **`liberaPosto` lancia eccezione se il contatore è già al massimo.** *Gestione:* l'ho tenuta apposta, come scelta "fail fast": meglio un crash che si vede subito che un contatore che va silenziosamente oltre la capacità (violando l'invariante di safety). Ho comunque coperto questo caso con un test dedicato.

4. **Scostamento temporale nel decremento dei contatori rispetto all'ASM:** `assegnaPosto` decide e decrementa nello stesso passo, mentre l'ASM decrementa al transito (INGR). *Gestione:* la differenza non si vede in nessuno scenario di test, quindi ho tenuto la versione più semplice e l'ho spiegata nel Javadoc della classe.
5. **Assenza di thread-safety.** *Gestione:* accettata, perché c'è un solo varco pilotato da una FSM sequenziale, come nel modello ASM; l'ho annotata come limite per eventuali estensioni multi-varco.
6. **Memoria a slot singolo per `posto_assegnato`, difetto presente anche nel modello ASM.** È l'osservazione più rilevante. Sia la FSM Java sia l'ASM memorizzano un solo posto assegnato, ma il parcheggio può contenere due veicoli (1 STD + 1 DIS). Se entra uno STD, poi un DISABLE (che sovrascrive lo slot), e infine esce il primo veicolo, viene rilasciato il posto sbagliato e i contatori si corrompono. *Gestione:* l'implementazione è conforme alla specifica; è la specifica stessa a presupporre implicitamente “un veicolo tracciato per volta”, senza un'associazione veicolo→posto. L'ho documentato come limite noto della specifica: correggerlo richiederebbe una mappa veicolo→posto a entrambi i livelli. È un esempio concreto di difetto che il testing di conformità non può rilevare (il codice rispetta il modello) e che emerge solo dall'ispezione critica.
7. **Bottoni della UI sempre attivi:** `MainView` non disabilita i comandi in base allo stato (si può cliccare “Pagamento” in IDLE). Gli step spuri risultano però innocui perché assorbiti dai self-loop della FSM, cioè dagli stessi `if` senza `else` segnalati dal Model Advisor come MP2 (§2.3), che questa osservazione conferma a posteriori come scelta difensiva corretta. *Gestione:* accettata per un prototipo; ho annotato la soluzione (`setEnabled` contestuale) per una versione reale.

6.3 Bilancio dell'ispezione

In tutto sono uscite 7 osservazioni: nessun bug vero da correggere nel codice (in due casi mi è bastato chiarire le cose nel Javadoc), ma è saltato fuori un **difetto latente nella specifica stessa** (l'osservazione 6), che né il testing di conformità né la verifica ESC riuscivano a vedere. I contratti JML di `Parcheggio` restano infatti rispettati anche nell'esecuzione che rilascia il posto sbagliato, perché il problema sta a monte, a livello di FSM e di modello. Per me è la conferma che questi livelli di verifica si completano a vicenda: il model checking e l'ESC dimostrano quello che è formalizzato, mentre l'ispezione critica trova proprio quello che la formalizzazione si lascia sfuggire.

7 Continuous Integration

Vediamo infine come ho impostato la continuous integration sul progetto. L'idea è che, ogni volta che si fa un push o si apre una pull request, i test partano da soli, così se qualcuno rompe qualcosa ce ne accorgiamo subito. Il tutto sta nel workflow `.github/workflows/ci.yml`: fa la build Maven completa (lasciando fuori i test taggati `ui`, che hanno bisogno di Chrome e dell'app avviata) e alla fine salva il report JaCoCo come artifact.

Listing 15: `.github/workflows/ci.yml` (estratto)

```

1 on:
2   push:
3   pull_request:
4
5 jobs:
6   build-and-test:
7     runs-on: ubuntu-latest

```



```

8  steps:
9    - uses: actions/checkout@v4
10   - uses: actions/setup-java@v4
11     with:
12       distribution: temurin
13       java-version: '17'
14   - run: mvn -B -f java/pom.xml verify
15   - uses: actions/upload-artifact@v4
16     with:
17       name: jacoco-report
18       path: java/target/site/jacoco/

```

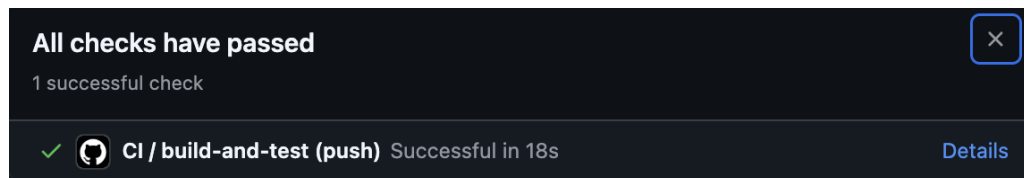
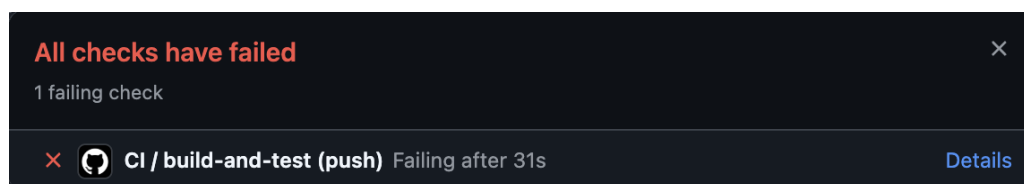


Figure 22: Esecuzione CI riuscita sul branch main

Avere la pipeline verde però non dimostra granché di per sé: dimostra che i test passano, non che la CI se ne accorgerebbe se smettessero di passare. Per verificare che il meccanismo funzionasse davvero ho quindi introdotto una regressione di proposito: su un branch dedicato (`ci_failure`, che ho lasciato apposta sul repository come evidenza) ho modificato `assegnaPosto` facendogli restituire sempre `TipoPosto.NESSUNO`. Al push la pipeline è andata in rosso, con i test di `ParcheggioTest` e `SmartParkingFSMTest` falliti sugli assert sull'esito dell'assegnazione. Il branch `main` è rimasto intatto.

Figure 23: Esecuzione CI fallita sul branch `ci_failure` (regressione introdotta di proposito)

Questo è, secondo me, il vero valore della continuous integration in un progetto come questo: non tanto far girare i test (posso farlo anche in locale), quanto avere la garanzia che non me ne possa dimenticare, e che ogni commit resti legato a una prova oggettiva del fatto che il codice funzionava in quel momento.

8 Conclusioni

Tirando le somme, il progetto tocca tutte e quattro le parti richieste: i requisiti, il modello Asmeta (con gli scenari, sia quelli corretti che quelli nuovi, e il model checking), i contratti JML sul nucleo del dominio e l'implementazione Java con i vari test (JUnit, MC/DC, Mockito, model based testing e combinatorial testing), la UI Vaadin, la CI e l'analisi statica.

8.1 Riepilogo delle evidenze

- Simulatore e animatore Asmeta (Sezione 2)
- AsmetaV – copertura degli scenari Avalla (Sezione 2)
- AsmetaSMV – 8 CTLSPEC verificate e 2 volutamente false con controesempio (Sezione 2)

- Model Advisor e ATGT, opzionali (Sezione 2)
- OpenJML ESC sui contratti del nucleo (Sezione 3)
- UI Vaadin in esecuzione e test Selenium (Sezione 4)
- JUnit (59 test del package `core`, tutti verdi, + 1 test Selenium UI escluso dalla build standard) e copertura JaCoCo (Sezione 5)
- Analisi statica e ispezione critica con LLM (Sezione 6)
- GitHub Actions: build e test automatici ad ogni push (Sezione 7)

Quello che mi piace di questo percorso è che si tiene tutto insieme: si parte dal modello astratto, lo si valida e lo si verifica, poi si passa ai contratti JML dimostrati staticamente e si finisce con i test sul codice e la CI. Alla fine si vede come le tecniche viste a lezione, ognuna con il suo scopo, entrino a far parte di un unico processo di sviluppo in cui a ogni livello si controlla qualcosa.